DSA STUDY NOTES

TWO SUM: BRUTE FORCE • HASHMAP • TWO POINTERS

Complete Java notes with concept clarity, full programs, dry runs, complexity analysis, common mistakes and practice.

Is notes ka aim sirf code yaad karna nahi hai. Aim hai samajhna ki har approach ka logic kya hai, loop/pointer kaise move karta hai, aur kis condition mein kaunsi approach use karni chahiye.

Study roadmap

Step	Topic	What you should understand
1	Brute Force & Linear Search	Possible options ko systematically check karna.
2	Two Sum problem	Do different indices ke elements ka sum target ke equal.
3	Nested loops	i first element choose karta hai; j uske baad wale elements.
4	HashMap	Past values store; current value ka complement search.
5	Two Pointers	Sorted array mein left/right pointers move karna.
6	Comparison	Time complexity, extra space, and input conditions.

How to study: Pehle concept padho, phir code ko line-by-line dekho, phir dry run khud notebook mein banao. Last mein practice questions solve karo.

1. Foundation: Brute Force

Definition: Brute Force ek problem-solving technique hai jisme hum possible candidates/options ko systematically check karte hain, jab tak answer mil na jaaye ya saare options check na ho jaayein.

Simple example: Agar ek bag mein 10 numbered cards hain aur tumhe number 7 dhoondhna hai, toh ek-ek card check karna ek straightforward brute-force approach hai.

Important distinction: Linear Search brute force ka ek example hai. Brute Force ek broad technique hai; Linear Search uska ek specific algorithm hai.

Linear Search example

Array: **[4, 9, 2, 7]**, Target: **2**
Check 4 → not equal; check 9 → not equal; check 2 → found at index 2.

Case	Meaning	Time
Best case	Target first position par mil gaya.	O(1)
Average case	Aam taur par array ka kuch hissa check hota hai.	O(n)
Worst case	Target last par ho ya array mein ho hi nahi.	O(n)

n ka matlab array mein total elements ki count hai. O(n) ka matlab input size badhne par work roughly linearly badhta hai.

2. Two Sum problem: Understand the question

Problem: Ek integer array **arr** aur ek integer **target** diya hai. Do different indices find karo jinke elements ka sum target ke equal ho.

Example: arr = [5, 2, 8, 5, 3, 7], target = 10.
arr[0] + arr[3] = 5 + 5 = 10. Answer indices: [0, 3].

> **Key point: Values same ho sakti hain, lekin indices different hone chahiye. Is example mein dono values 5 hain, par positions 0 aur 3 alag hain.**

Index aur value ka difference

Index	0	1	2	3	4	5
Value	5	2	8	5	3	7

3. Approach 1 — Brute Force (Nested Loops)

Logic: Outer loop ka i ek element choose karta hai. Inner loop ka j, i ke baad ke elements check karta hai. Har possible pair check hota hai.

Why j = i + 1?
• Same element ko khud ke saath pair nahi karna.
• Pehle check kiya hua pair reverse order mein dobara check nahi karna.
• Isliye j, i ke next index se start hota hai.

Complete Java program

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {5, 2, 8, 5, 3, 7};
        int target = 10;

        boolean found = false;

        for (int i = 0; i < arr.length; i++) {
            for (int j = i + 1; j < arr.length; j++) {
                if (arr[i] + arr[j] == target) {
                    System.out.println("Indices: " + i + ", " + j);
                    System.out.println("Values: "
                        + arr[i] + " + " + arr[j]);
                    found = true;
                    break;
                }
            }
            if (found) {
                break;
            }
        }

        if (!found) {
            System.out.println("Pair not found");
        }
    }
}
```

Expected output:
Indices: 0, 3
Values: 5 + 5

Dry run: arr = [2, 7, 11, 15], target = 9

i	j	Pair	Sum	Result
0	1	2 + 7	9	Pair found; stop.

Complexity: Time $O(n^2)$, because nested loops can examine many pairs. Extra space O(1), because the algorithm uses only a fixed number of extra variables.

4. Approach 2 — HashMap

Main idea: Har step par current value ke saath pair banane ke liye humein ek required value chahiye. Formula:

> **required = target - current value**

HashMap mein hum pehle dekhe gaye actual array values ko key aur unke index ko value ke roop mein store karte hain.

Name	Meaning
current	arr[i] — jo value abhi process kar rahe hain.
required	target - current — jo doosri value humein chahiye.
map	Ab tak dekhi hui values aur unke indices.
containsKey(x)	Check karta hai ki key x map mein exist karti hai ya nahi.
get(x)	Key x se associated index/value return karta hai.
put(x, i)	Map mein key x ke saath index i store karta hai.

Complete Java program

```
import java.util.HashMap;
import java.util.Map;

public class Main {
    public static void main(String[] args) {
        int[] arr = {2, 7, 11, 15};
        int target = 9;

        Map<Integer, Integer> map = new HashMap<>();

        for (int i = 0; i < arr.length; i++) {
            int current = arr[i];
            int required = target - current;

            if (map.containsKey(required)) {
                int previousIndex = map.get(required);
                System.out.println("Indices: "
                    + previousIndex + ", " + i);
                return;
            }

            map.put(current, i);
        }

        System.out.println("Pair not found");
    }
}
```

Expected output: Indices: 0, 1

HashMap dry run — line by line

i	current	required = 9-current	Map before lookup	Action
0	2	7	{}	7 absent. Store 2 → 0. Map becomes {2=0}.
1	7	2	{2=0}	2 exists. get(2) gives index 0. Answer [0,1].

Why do we store current, not required?

Is version mein map ka role pehle dekhe gaye actual array numbers ko remember karna hai. Har naye current number

ke liye required complement map mein search hota hai. Isliye hum current ko store karte hain.

Important: HashMap mein keys unique hoti hain. Same key dobara put karoge toh us key ki old value overwrite ho sakti hai. Is basic Two Sum code mein answer milte hi return kar dete hain.

Complexity: Average time O(n), extra space O(n). Har element par average O(1) HashMap lookup/insert hota hai. HashMap use karne se logic automatically correct nahi ho jaata; correct implementation zaroori hai.

5. Approach 3 — Two Pointers

Condition: Standard Two Sum Two-Pointers method ke liye array sorted hona chahiye (ascending order).

Do pointers banao: **left = 0** (first index), **right = arr.length - 1** (last index). Dono pointed values ka sum calculate karo.

Condition	What to do	Reason
sum == target	Answer found.	Required sum mil gaya.
sum < target	left++	Bigger value ki taraf move karna hai.
sum > target	right--	Smaller value ki taraf move karna hai.

Remember: Indices ko add/subtract nahi karte. left aur right pointers ko move karte hain.

Complete Java program

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {1, 2, 4, 6, 8}; // sorted array
        int target = 10;

        int left = 0;
        int right = arr.length - 1;

        while (left < right) {
            int sum = arr[left] + arr[right];

            if (sum == target) {
                System.out.println("Indices: "
                    + left + ", " + right);
                System.out.println("Values: "
                    + arr[left] + " + " + arr[right]);
                return;
            } else if (sum < target) {
                left++;
            } else {
                right--;
            }
        }

        System.out.println("Pair not found");
    }
}
```

Expected output:
Indices: 1, 4
Values: 2 + 8

Two Pointers dry run

Step	left value	right value	sum	Decision
1	1	8	9	9 < 10, so left++
2	2	8	10	10 = 10, found indices [1,4]

Why does the pointer rule work?
Sorted ascending array mein left ko aage badhane se left value badi hoti hai. Right ko peeche laane se right value

chhoti hoti hai. Isliye sum chhota ho toh left move karte hain; sum bada ho toh right move karte hain.

Warning: Unsorted array par ye pointer decisions reliable nahi hote. Agar original unsorted array ko sort karte ho, toh original indices preserve karne ke liye value-index pairs track karne pad sakte hain.

Complexity: Time O(n), extra space O(1), assuming array already sorted hai. Agar sorting karni pade toh sorting ka cost separately count hoga.

6. Compare all three Two Sum approaches

Approach	Time	Extra space	Input condition	Core idea
Brute Force	$O(n^2)$	O(1)	Any order	Har pair check karo.
HashMap	O(n) average	O(n)	Any order	Past values store; complement search.
Two Pointers	O(n)	O(1)	Sorted array	Left/right pointer move.

Which one should you choose?
• Beginner logic samajhne ke liye: Brute Force.
• Unsorted array mein fast lookup aur indices chahiye: HashMap.
• Array already sorted ho aur extra memory kam rakhni ho: Two Pointers.

7. Common mistakes — avoid these

• **Mistake 1:** Two Sum mein same index ko do baar use karna. Brute Force mein j = i + 1 use karo.

• **Mistake 2:** HashMap mein required ko store karna, jab tum current-value storage wali implementation follow kar rahe ho. Is version ka rule: search required, store current.

• **Mistake 3:** containsKey() aur get() ko same samajhna. containsKey existence check karta hai; get associated value deta hai.

• **Mistake 4:** Two Pointers ko unsorted array par directly use karna. Standard method sorted input maangta hai.

• **Mistake 5:** left++ ko index addition samajhna. Ye pointer ko next position par move karta hai.

• **Mistake 6:** “Optimal approach never fails” sochna. Optimal ka matlab complexity behtar; correctness conditions aur code par depend karti hai.

8. Quick revision sheet

Concept	One-line memory rule
Brute Force	Possible candidates ko systematically check karo.
Linear Search	Elements ko one by one check karo.
Two Sum	Two different indices; values ka sum = target.
Nested loops	i first; j = i + 1 next elements.
HashMap	Key = previous actual value; Value = its index.
Complement	required = target - current.
Two Pointers	sum small → left++; sum large → right--.
Sorted condition	Two Pointers ka standard rule sorted array par depend karta hai.

9. Practice questions

Q1. arr = [3, 4, 6, 8], target = 10. Brute Force se indices find karo.

Q2. Same input par HashMap ka dry run banao. Har iteration mein current, required aur map likho.

Q3. Sorted arr = [1, 3, 5, 7, 9], target = 12. Two Pointers ke left/right movements trace karo.

Q4. Two Pointers method ke liye sorted array kyun important hai?

Q5. Brute Force $O(n^2)$, HashMap average O(n), Two Pointers O(n): inke extra space ka comparison likho.

In questions ko bina notes dekhe solve karne ki koshish karo. Agar dry run khud bana pa rahe ho, toh concept samajh aa raha hai.